plan-v3-analista-datos-hackaton

🤖 Agente de Análisis de Datos — Plan de Proyecto v3

Hackaton Microsoft Azure · Semana del 30 de marzo · Equipo 3–4 personas

---

📋 Resumen ejecutivo

Sistema de IA multi-agente que convierte preguntas en lenguaje natural en consultas SQL validadas, las ejecuta de forma segura contra bases de datos empresariales y explica los resultados en lenguaje de negocio claro. Diseñado para funcionar con cualquier empresa que conecte su base de datos, con trazabilidad completa, seguridad activa demostrable y 13 servicios entre Azure y servicios complementarios.

Campo	Detalle
Stack backend	Python · FastAPI · Azure Container Apps
Stack frontend	React · Next.js · Chart.js
IA principal	Azure OpenAI GPT-4o
Framework agentes	Semantic Kernel + LangGraph (Agente 2)
Base de conocimiento	Azure AI Search (RAG con 2 libros)
Acceso a datos	Data API Builder (DAB) con permisos por usuario
Autenticación	Firebase Auth
Guardrails	Prompt Shields (Azure AI Content Safety)
Seguridad activa	Dashboard de amenazas detectadas y prevenidas
Variables de entorno	.env por empresa (sin Key Vault)
Fecha de presentación	Semana del 30 de marzo

🏆 Criterios de evaluación y cómo los cubrimos

Criterio	Peso	Cómo lo demostramos
Rendimiento	25%	Auto-corrección de SQL con LangGraph · esquema indexado en AI Search · DAB como capa de acceso rápida · Circuit Breaker para resiliencia
Innovación	25%	Onboarding automático de esquema · RAG con libros técnicos · insights con recomendaciones técnicas · permisos DAB por usuario
Amplitud de Azure	25%	13 servicios activos y demostrables en el flujo
IA Responsable	25%	Prompt Shields · filtros de contexto · dashboard de amenazas detectadas · trazabilidad completa · auditoría exportable

---

🏗️ Arquitectura del sistema

https://claude.ai/public/artifacts/daf0f937-c68e-4e72-bbb9-36427585e222

Flujo principal

```
Usuario empresarial
      ↓
Frontend Next.js + React
(chat · visualizaciones · panel de auditoría · dashboard de seguridad)
      ↓
Azure API Management
(routing multi-tenant · rate limiting)
      ↓
Firebase Auth
(autenticación · roles por empresa · token con tenant_id y permisos DAB)
      ↓
Backend FastAPI — Azure Container Apps
(orquestador con Semantic Kernel)
      ↓
┌──────────────────────────────────────────────────────────────────────┐
│            CAPA DE 4 AGENTES (GPT-4o)            │
│                                                  │
│  [1] Agente de intención analítica               │
│      ↓ consulta RAG (libros + esquema DAB)       │
│  [2] Agente generador de SQL  ← LangGraph        │
│      ↓ Prompt Shields + filtros de contexto      │
│  [3] Agente de ejecución segura                  │
```

```
│      ↓ via DAB (permisos por usuario) + CB          │
│  [4] Agente de insights y recomendaciones           │
│      ↓ consulta RAG libro de Cole                   │
└──────────────────────────────────────────────────────┘
      ↓
Data API Builder → Azure SQL / Synapse Analytics
      ↓
Respuesta con insight + recomendaciones + chart + fuente
      ↓
Dashboard de seguridad (amenazas detectadas y prevenidas)
```

🔧 Los 13 servicios y su rol

#	Servicio	Rol en el sistema	Criterio
1	Azure OpenAI GPT-4o	Motor de los 4 agentes	Rendimiento + Innovación
2	Azure OpenAI Embeddings (text-embedding-3-large)	Vectorización RAG	Rendimiento
3	Azure AI Search	RAG libros + esquema BD	Innovación + Amplitud
4	Data API Builder (DAB)	Capa REST/GraphQL con permisos por usuario	Innovación + Amplitud
5	Azure SQL Database	Ejecución de consultas (demo Contoso)	Rendimiento
6	Azure Synapse Analytics	Ejecución producción / data lake	Amplitud
7	Firebase Auth	Autenticación · roles · token con permisos DAB	Amplitud
8	Azure API Management	Gateway multi-tenant · rate limiting	Amplitud + Rendimiento
9	Prompt Shields (Azure AI Content Safety)	Guardrail contra inyección y document attacks	IA Responsable
10	Azure App Insights + OpenTelemetry	Trazas distribuidas por agente	IA Responsable + Rendimiento
11	Azure Container Apps	Deploy backend Python + DAB	Amplitud

12	Azure Blob Storage	PDFs de libros + logs de auditoría + .env por empresa	Amplitud
13	LangGraph	Orquestación de ciclos de auto-corrección del Agente 2	Rendimiento + Innovación

---

🤖 Los 4 agentes en detalle

Agente 1 — Descomposición de intención analítica

- **Input:** pregunta en lenguaje natural del usuario
- **Proceso:** identifica tablas, métricas, filtros, período y técnica SQL · consulta RAG libros para técnica avanzada · verifica tablas disponibles contra esquema DAB · si la pregunta es ambigua solicita clarificación
- **Output:** JSON `{ tablas, metricas, filtros, periodo, tecnica_sugerida, clarificacion_requerida }`

Agente 2 — Generador y validador de SQL (LangGraph)

- **Input:** JSON del Agente 1 + esquema de la empresa desde DAB
- **Proceso:** LangGraph orquesta el ciclo: genera SQL → valida sintaxis con `sqlparse` → Prompt Shields detecta ataques → filtro de contexto verifica que la consulta esté dentro del dominio permitido → auto-corrige si hay error (máx. 3 intentos con backoff)
- **Restricción fuerte:** nunca genera DELETE, UPDATE, DROP, INSERT, CREATE, TRUNCATE
- **Filtro de contexto:** verifica que las columnas solicitadas correspondan al rol del usuario según los permisos del token de Firebase

Agente 3 — Ejecución segura via DAB con Circuit Breaker

- **Input:** SQL validado y aprobado por los filtros
- **Proceso:** Circuit Breaker verifica disponibilidad de DAB → llama a API REST de DAB → DAB aplica permisos por usuario declarados en `dab-config.json` → timeout 30s · límite 10.000 filas · si DAB retorna error lo pasa al Agente 2 para auto-corrección
- **Circuit Breaker:** 5 fallos consecutivos abren el circuito · timeout 60s antes de reintentar · mensaje claro al usuario si BD no disponible

Agente 4 — Generador de insights y recomendaciones

- **Input:** DataFrame + pregunta original + contexto de la empresa
- **Proceso:** consulta libro de Cole (RAG filtrado por `tipo_uso: visualizacion`) · genera resumen de negocio · interpreta los datos identificando tendencias,

anomalías y correlaciones · entrega recomendaciones en lenguaje técnico-empresarial medio
- **Output:** resumen textual · hallazgos no obvios · recomendaciones accionables · tipo de chart · datos para Chart.js · fuente citada (libro + capítulo + página)

🔐 Firebase Auth — autenticación y permisos DAB

Firebase reemplaza Azure Entra ID. El token de Firebase incluye claims personalizados que definen el `tenant_id` y el rol del usuario, los cuales DAB usa para aplicar permisos granulares.

Roles y permisos

```
// Custom claims en Firebase por usuario
{
  "tenant_id": "empresa_a",
  "role": "analyst",          // analyst · manager · admin
  "allowed_tables": ["Sales", "Customers", "Products"],
  "restricted_columns": ["salary", "ssn", "bank_account"]
}
```

Configuración DAB con permisos por rol

```
{
  "entities": {
    "Employees": {
      "source": { "object": "dbo.Employees", "type": "table" },
      "rest": { "enabled": true },
      "permissions": [
        {
          "role": "analyst",
          "actions": ["read"],
          "fields": {
            "include": ["id", "name", "department", "position"],
            "exclude": ["salary", "ssn", "bank_account"]
          }
        },
        {
          "role": "manager",
          "actions": ["read"],
          "fields": { "include": ["*"] }
        }
      ]
    }
  }
```

}

Esto significa que un analista que pregunta “¿cuáles son los salarios del equipo?” recibe automáticamente una respuesta filtrada sin esos campos — DAB los excluye antes de retornar los datos.

🛡️ IA Responsable — filtros de contexto y dashboard de seguridad

Filtros de seguridad activos (en orden de ejecución)

Filtro 1 — Prompt Shields: detecta user prompt attacks y document attacks antes de generar SQL.

Filtro 2 — Filtro de contexto empresarial: verifica que la pregunta esté dentro del dominio de datos de la empresa. Una empresa de ventas no debería poder preguntar sobre nóminas si no tiene esa tabla configurada.

```
def context_filter(question, tenant_schema, user_role):
    # Verifica que las entidades identificadas existen en el esquema del tenant
    # Verifica que el rol del usuario tiene acceso a esas entidades
    # Bloquea preguntas fuera del dominio configurado
    allowed_entities = get_allowed_entities(tenant_schema, user_role)
    detected_entities = extract_entities(question)
    unauthorized = [e for e in detected_entities if e not in allowed_entities]
    if unauthorized:
        return BlockedResult(reason="out_of_context", entities=unauthorized)
```

Filtro 3 — Interpretador semántico de riesgo: analiza las columnas del SELECT contra el catálogo de campos sensibles, estima volumen potencial del resultado y detecta JOINs que cruzan dominios sensibles.

Filtro 4 — Circuit Breaker: evita saturar la BD cuando está fallando.

Dashboard de seguridad

El dashboard reemplaza el panel de RAI Toolbox genérico. Muestra en tiempo real:

Métrica	Descripción
Amenazas bloqueadas hoy	Contador de Prompt Shields activados
Consultas fuera de contexto	Preguntas bloqueadas por el filtro contextual

Intentos de acceso a columnas restringidas	Usuarios que intentaron ver datos sin permiso
Circuit Breaker activaciones	Veces que la BD no estaba disponible
Tipo de ataque más frecuente	Clasificación de amenazas (encoding · role-play · injection)
Tasa de bloqueo por empresa	Comparación entre tenants

Todo conectado al panel de auditoría que muestra el historial exportable en CSV.

📚 Base de conocimiento RAG

Libros indexados

1. **SQL for Data Analysis** — Cathy Tanimura · `tipo_uso: generacion_sql` · Agentes 1 y 2
2. **Storytelling with Data** — Cole Nussbaumer · `tipo_uso: visualizacion` · Agente 4

Cómo lo usan los agentes en código

```
from azure.search.documents import SearchClient
from azure.search.documents.models import VectorizedQuery

def search_knowledge_base(query: str, tipo_uso: str, tenant_id: str):
    search_client = SearchClient(
        endpoint=AZURE_SEARCH_ENDPOINT,
        index_name="books-index",
        credential=AzureKeyCredential(AZURE_SEARCH_KEY)
    )
    vector_query = VectorizedQuery(
        vector=get_embedding(query),
        k_nearest_neighbors=5,
        fields="vector"
    )
    results = search_client.search(
        search_text=query,
        vector_queries=[vector_query],
        filter=f"tipo_uso eq '{tipo_uso}'",
        select=["contenido", "libro", "capitulo", "pagina"],
        top=5
    )
    return list(results)
```

🔄 LangGraph — ciclo de auto-corrección del Agente 2

LangGraph reemplaza la lógica de reintentos manual. Define el ciclo de generación como un grafo de estados:

```
[GENERAR SQL]
    ↓
[VALIDAR SINTAXIS]
    ↓ falla
[ANALIZAR ERROR] → [REGENERAR SQL] → vuelve a VALIDAR
    ↓ pasa
[PROMPT SHIELDS]
    ↓ bloqueado → registrar en dashboard de seguridad → FIN
    ↓ pasa
[FILTRO DE CONTEXTO]
    ↓ bloqueado → registrar en dashboard de seguridad → FIN
    ↓ pasa
[SQL LISTO PARA EJECUTAR]
```

Máximo 3 ciclos de regeneración. Si supera los 3, escala al usuario con un mensaje explicando qué parte de la pregunta no pudo resolverse.

📅 Plan de desarrollo (17–29 de marzo)

Días	Fechas	Foco	Entregable
1–2	17–18 mar	Infraestructura + DAB	Container Apps · .env configurado · DAB corriendo
3–4	19–20 mar	Firebase Auth + multi-tenant	Login · roles · permisos DAB por usuario
5–6	21–22 mar	RAG + libros	Libros indexados · búsqueda híbrida · API AI Search en código
7–8	23–24 mar	Agentes 1 y 2 + Prompt Shields	Intención → SQL · LangGraph · guardrails activos
9–10	25–26 mar	Agente 3 (DAB + CB) + Agente 4	Ejecución segura · insights · recomendaciones
11–12	27–28 mar	Dashboard seguridad + auditoría	Filtros de contexto · métricas de amenazas · panel completo

13	29 mar	Demo + presentación	Script ensayado · Contoso data · pitch 5 min

---

📦 Módulos e Issues

M1 — Infraestructura y configuración base

> Azure Container Apps · CI/CD · .env por empresa · App Insights · OpenTelemetry

Estimado total: 10 horas

#01 — Configurar repositorio GitLab con estructura de proyecto

Tipo: `infra` · **Prioridad:** `alta` · **Estimado:** 2h

Acceptance criteria:

- [ ] El repositorio tiene carpetas `/backend`, `/frontend`, `/infra`, `/dab`, `/docs` claramente separadas
- [ ] La carpeta `/dab` contiene los archivos `dab-config.json` por empresa de demo
- [ ] Existe un `.env.example` con todas las variables necesarias sin valores reales
- [ ] El `.gitignore` excluye `.env`, `__pycache__`, `node_modules` y cualquier archivo con credenciales
- [ ] El pipeline de CI básico (GitLab CI) corre lint y tests al hacer push a `main`
- [ ] El `README.md` documenta cómo configurar el `.env` local paso a paso

Definition of done: Pipeline de CI corriendo sin errores · `.env.example` completo · estructura validada por el equipo

---

#02 — Provisionar Azure Functions para backend FastAPI y DAB

Tipo: `infra` · **Prioridad:** `alta` · **Estimado:** 4h **Acceptance criteria:**

- [ ] La Function App de FastAPI responde en `/health` con status 200 desde internet
- [ ] DAB corre en una Function App separada con endpoint `/health` activo
- [ ] El deploy se hace automáticamente desde GitLab CI al hacer push a `main`

- [ ] Las variables de entorno se inyectan desde el `.env` configurado en la Function App — sin Key Vault
- [ ] Las functions escalan a 0 instancias cuando no hay tráfico para optimizar costos
- [ ] Existe un entorno de staging separado para pruebas antes de producción

Definition of done: Endpoints `/health` de FastAPI y DAB accesibles · deploy automático en menos de 5 minutos · variables de .env correctamente inyectadas

---

#03 — Configurar Azure Monitor + OpenTelemetry para trazabilidad distribuida

Tipo: `infra` · **Prioridad:** `media` · **Estimado:** 4h

Acceptance criteria:

- [ ] El SDK de OpenTelemetry está configurado en Python apuntando a App Insights
- [ ] Cada llamada a los 4 agentes genera un span individual con duración y metadata
- [ ] Los errores de SQL incluyen el mensaje de error y el SQL que falló como atributos del span
- [ ] Los bloqueos de Prompt Shields y filtros de contexto generan eventos en App Insights
- [ ] Las activaciones del Circuit Breaker generan alertas en App Insights
- [ ] Las trazas incluyen `tenant_id` y `user_role` para filtrar por empresa y perfil

Definition of done: Dashboard de App Insights con spans por agente visibles · eventos de seguridad registrados · filtros por tenant_id operativos

---

M2 — Autenticación Firebase y multi-tenant

> Firebase Auth · custom claims · permisos DAB por usuario · aislamiento por empresa

Estimado total: 14 horas

#04 — Integrar Firebase Auth con custom claims para roles y permisos

Tipo: `seguridad` · **Prioridad:** `alta` · **Estimado:** 5h

Acceptance criteria:

- [ ] El usuario inicia sesión con email/password o Google vía Firebase Auth

- [ ] El token de Firebase incluye custom claims: `tenant_id`, `role`, `allowed_tables`, `restricted_columns`
- [ ] El backend verifica el token de Firebase en cada request usando el SDK de Firebase Admin
- [ ] Las rutas del backend devuelven 401 sin token válido y 403 si el rol no tiene acceso
- [ ] El frontend redirige al login de Firebase si no hay sesión activa
- [ ] El logout invalida la sesión y limpia el token del frontend

Definition of done: Login funcional con Firebase · custom claims en el token verificados en backend · 401 y 403 funcionando correctamente

---

#05 — Configurar DAB con permisos granulares por rol de usuario

Tipo: `infra` · **Prioridad:** `alta` · **Estimado:** 5h

Acceptance criteria:

- [ ] DAB está instalado con `dotnet tool install microsoft.dataapibuilder -g`
- [ ] Cada empresa tiene su `dab-config.json` con permisos diferenciados por rol (`analyst`, `manager`, `admin`)
- [ ] El rol `analyst` tiene acceso de solo lectura y con columnas sensibles excluidas (salary, ssn, etc.)
- [ ] El rol `manager` tiene acceso de lectura completo a todas las columnas
- [ ] El rol `admin` puede ver el esquema completo incluyendo metadata
- [ ] Un intento de escritura via DAB retorna 403 independientemente del rol
- [ ] DAB corre en contenedor en Azure Container Apps con el `.env` como fuente de credenciales

Definition of done: Prueba de acceso cruzado confirmada · columnas sensibles no visibles para `analyst` · intento de escritura retorna 403

#06 — Implementar aislamiento de datos por empresa (multi-tenant)

Tipo: `seguridad` · **Prioridad:** `alta` · **Estimado:** 4h

Acceptance criteria:

- [ ] Cada empresa tiene su propio `dab-config.json` con su cadena de conexión en el `.env`
- [ ] Un usuario de empresa A no puede acceder a datos de empresa B bajo ninguna circunstancia

- [ ] El esquema indexado en AI Search está particionado por campo `tenant_id`
- [ ] DAB usa el `tenant_id` del token de Firebase para enrutar al contenedor correcto
- [ ] Existe prueba automatizada que verifica el aislamiento entre dos tenants

Definition of done: Prueba cruzada confirmada: usuario empresa A no ve datos empresa B · aislamiento en DAB y AI Search verificado · test automatizado pasando

---

#07 — Crear flujo de onboarding automático para nueva empresa

Tipo: `feature` · **Prioridad:** `media` · **Estimado:** 5h

Acceptance criteria:

- [ ] La empresa ingresa su cadena de conexión en formulario seguro del frontend
- [ ] El sistema genera automáticamente el `dab-config.json` con las tablas y columnas descubiertas
- [ ] El sistema detecta automáticamente columnas sensibles y las excluye del rol `analyst`
- [ ] DAB se reinicia con la nueva configuración en menos de 2 minutos
- [ ] El esquema se indexa en AI Search en menos de 5 minutos desde el onboarding
- [ ] El sistema genera 3 preguntas de ejemplo basadas en el esquema real de la empresa

Definition of done: Demo con dos empresas onboardeadas exitosamente · columnas sensibles detectadas y excluidas automáticamente · preguntas de ejemplo relevantes

---

M3 — Base de conocimiento RAG

Azure AI Search · libros técnicos · esquema dinámico vía DAB · API en código

Estimado total: 11 horas

#08 — Subir y configurar indexación de los dos libros PDF

Tipo: `feature` · **Prioridad:** `alta` · **Estimado:** 4h

Acceptance criteria:

- [ ] Los dos PDFs están en Azure Blob Storage en contenedor privado
- [ ] El Indexer de AI Search procesa los PDFs con OCR y chunking por secciones del libro

- [ ] Cada chunk incluye: `libro`, `capitulo`, `pagina`, `tipo_uso` (`generacion_sql` o `visualizacion`)
- [ ] La indexación completa de ambos libros termina en menos de 15 minutos
- [ ] El libro de Tanimura tiene `tipo_uso: generacion_sql` y el de Cole `tipo_uso: visualizacion`

Definition of done: 10 queries de prueba retornan el libro correcto · metadata correcta en cada chunk · tiempo de indexación verificado

---

#09 — Integrar búsqueda híbrida de AI Search directamente en el código de los agentes

Tipo: `feature` · **Prioridad:** `alta` · **Estimado:** 3h

Acceptance criteria:

- [ ] Los agentes usan el SDK `azure-search-documents` directamente — sin capa intermedia
- [ ] La búsqueda combina vector (HNSW) + BM25 con RRF y Semantic Ranker
- [ ] El Agente 2 filtra por `tipo_uso: generacion_sql` para obtener solo técnicas SQL
- [ ] El Agente 4 filtra por `tipo_uso: visualizacion` para obtener solo guías de visualización
- [ ] El tiempo de recuperación de top-5 chunks es menor a 500ms en promedio
- [ ] Los chunks retornados incluyen siempre `libro`, `capitulo` y `pagina` para la cita

Definition of done: Agentes 2 y 4 usan AI Search directamente en código · citas correctas en las respuestas · tiempo bajo 500ms verificado

---

#10 — Indexar esquema de BD dinámicamente desde DAB

Tipo: `feature` · **Prioridad:** `alta` · **Estimado:** 4h

Acceptance criteria:

- [ ] El sistema extrae el esquema leyendo el `dab-config.json` y consultando los endpoints de DAB
- [ ] El esquema se indexa en AI Search con campo `tenant_id` para aislamiento
- [ ] Si el esquema cambia y DAB se regenera, el indexador actualiza solo los elementos modificados
- [ ] El Agente 1 recupera el esquema de la empresa en menos de 500ms
- [ ] El esquema incluye tablas, columnas, tipos, y qué columnas están restringidas por rol

Definition of done: Esquema indexado con info de permisos por rol · actualización incremental funcional · tiempo bajo 500ms

---

M4 — Agentes de IA (núcleo del sistema)

> 4 agentes GPT-4o · Semantic Kernel · LangGraph en Agente 2 · Prompt Shields · Circuit Breaker

Estimado total: 28 horas

#11 — Implementar Agente 1 — Descomposición de intención analítica

Tipo: `feature` · **Prioridad:** `alta` · **Estimado:** 5h

Acceptance criteria:

- [ ] El agente identifica correctamente las tablas involucradas en al menos el 90% de las pruebas
- [ ] El output es siempre JSON válido con: `tablas`, `metricas`, `filtros`, `periodo`, `tecnica_sugerida`
- [ ] El agente consulta el RAG de libros (AI Search SDK) para sugerir la técnica SQL apropiada
- [ ] Verifica contra el esquema DAB que las tablas existen Y que el usuario tiene acceso según su rol
- [ ] Preguntas ambiguas generan solicitud de clarificación con opciones concretas al usuario
- [ ] Si la pregunta está completamente fuera del dominio de datos, lo indica claramente

Definition of done: Suite de 20 preguntas con 90%+ de intención correcta · JSON válido 100% · verificación de permisos por rol funcional

---

#12 — Implementar Agente 2 — Generador SQL con LangGraph

Tipo: `feature` · **Prioridad:** `alta` · **Estimado:** 8h

Acceptance criteria:

- [ ] LangGraph define el grafo de estados: Generar → Validar → Prompt Shields → Filtro contexto → Listo
- [ ] El SQL generado es sintácticamente válido con `sqlparse` en el 100% de los casos

- [ ] El agente nunca genera DELETE, UPDATE, DROP, INSERT, CREATE ni TRUNCATE
- [ ] El ciclo de auto-corrección usa backoff exponencial entre reintentos (1s · 2s · 4s)
- [ ] Si supera 3 intentos fallidos, escala al usuario con explicación del problema específico
- [ ] El SQL incluye comentarios explicando cada parte principal de la consulta
- [ ] El SQL generado se muestra al usuario con explicación en lenguaje simple antes de ejecutar

Definition of done: 100% SQL read-only · grafo LangGraph con los estados definidos · auto-corrección funcional · backoff verificado

---

#13 — Integrar Prompt Shields como nodo del grafo LangGraph

Tipo: `seguridad` · **Prioridad:** `alta` · **Estimado:** 4h

Acceptance criteria:

- [ ] Prompt Shields es un nodo explícito en el grafo de LangGraph — no una validación externa
- [ ] **User Prompt attacks** bloqueados: override del sistema, encoding attacks, role-play, conversational mockup
- [ ] **Document attacks** bloqueados: instrucciones ocultas en documentos subidos por usuarios
- [ ] SQL ofuscado en base64 o encoding alternativo es detectado y bloqueado
- [ ] Cada bloqueo genera evento en App Insights con: tipo de ataque · tenant_id · user_role · timestamp
- [ ] El evento de bloqueo alimenta automáticamente el dashboard de seguridad en el frontend
- [ ] El usuario recibe mensaje claro y amigable explicando por qué fue bloqueada la consulta

Definition of done: 15 intentos de ataque bloqueados al 100% · eventos en App Insights · dashboard actualizado en tiempo real · mensajes amigables al usuario

---

#14 — Implementar filtro de contexto empresarial como nodo LangGraph

Tipo: `seguridad` · **Prioridad:** `alta` · **Estimado:** 3h

Acceptance criteria:

- [ ] El filtro verifica que todas las tablas del SQL existen en el esquema del tenant
- [ ] El filtro verifica que el rol del usuario tiene acceso a todas las tablas y columnas del SQL

- [ ] Preguntas sobre tablas de otra empresa o fuera del esquema son bloqueadas con mensaje claro
- [ ] Preguntas que intentan acceder a columnas restringidas son bloqueadas y el bloqueo se registra
- [ ] Cada bloqueo por contexto genera evento en App Insights y alimenta el dashboard de seguridad
- [ ] El filtro usa el catálogo de columnas sensibles configurado en el `dab-config.json`

Definition of done: 10 casos de acceso fuera de contexto bloqueados correctamente · eventos en dashboard de seguridad · catálogo configurable funcional

---

#15 — Implementar Agente 3 — Ejecución segura via DAB con Circuit Breaker

Tipo: `feature` · **Prioridad:** `alta` · **Estimado:** 5h

Acceptance criteria:

- [ ] El agente llama a la API REST de DAB (`GET /api/{tabla}?$filter=...`) — nunca conexión directa a BD
- [ ] Circuit Breaker configurado: 5 fallos → circuito abierto · 60s timeout · reintento con consulta de prueba
- [ ] Cuando el circuito está abierto el usuario recibe mensaje claro: “Base de datos no disponible, intenta en X minutos”
- [ ] Las activaciones del Circuit Breaker generan evento en App Insights y en el dashboard de seguridad
- [ ] Las consultas tienen timeout de 30 segundos y límite de 10.000 filas
- [ ] Si DAB retorna error de datos, el mensaje se pasa al Agente 2 para auto-corrección
- [ ] El DataFrame resultado incluye metadata: tiempo de ejecución, filas retornadas, endpoint DAB usado

Definition of done: Circuit Breaker demostrable apagando DAB y verificando el mensaje al usuario · timeout funcional · retry con Agente 2 operativo

---

#16 — Implementar Agente 4 — Insights, interpretación y recomendaciones

Tipo: `feature` · **Prioridad:** `alta` · **Estimado:** 6h

Acceptance criteria:

- [ ] El resumen de negocio usa el lenguaje del dominio de la empresa, no terminología técnica de BD
- [ ] El agente **interpreta** los datos: identifica tendencias, anomalías, correlaciones y valores atípicos
- [ ] El agente entrega al menos 2 **recomendaciones accionables** en lenguaje técnico-empresarial medio
- [ ] El tipo de visualización está justificado con chunk del libro de Cole citado explícitamente
- [ ] La respuesta incluye siempre: resumen · hallazgos · recomendaciones · tipo de chart · fuente (libro+capítulo+página)
- [ ] El agente sugiere una pregunta de seguimiento relevante basada en los resultados obtenidos

Definition of done: 5 demos donde el agente identifica al menos 1 anomalía real · recomendaciones relevantes al dominio · citas del libro de Cole correctas

---

M5 — Seguridad, auditoría y dashboard de amenazas

> Filtros de contexto · auditoría exportable · dashboard de seguridad en tiempo real

Estimado total: 12 horas

#17 — Implementar interpretador semántico de riesgo de consultas

Tipo: `seguridad` · **Prioridad:** `media` · **Estimado:** 4h

Acceptance criteria:

- [ ] El interpretador analiza columnas del SELECT contra catálogo de campos sensibles configurable por empresa
- [ ] Detecta consultas sin WHERE sobre tablas grandes y las marca como riesgo alto
- [ ] Detecta JOINs que cruzan dominios sensibles distintos (ej. RRHH + finanzas)
- [ ] La clasificación combina 3 dimensiones: tipo de columnas · volumen estimado · dominios cruzados
- [ ] El resultado de la clasificación es visible al usuario: qué columnas sensibles encontró y por qué
- [ ] Los campos del catálogo sensible son editables por el admin de la empresa desde el frontend

Definition of done: 15 consultas clasificadas correctamente incluyendo casos donde el nombre de tabla no da pistas · catálogo configurable · clasificación visible al usuario

---

#18 — Implementar panel de auditoría completo

Tipo: `feature` · **Prioridad:** `media` · **Estimado:** 4h

Acceptance criteria:

- [ ] Cada consulta queda registrada con: pregunta · SQL generado · usuario · timestamp · estado · nivel de riesgo
- [ ] Los bloqueos de Prompt Shields incluyen el tipo de ataque detectado
- [ ] Los bloqueos del filtro de contexto incluyen qué tabla o columna fue el motivo
- [ ] Panel filtrable por fecha, usuario, estado, nivel de riesgo y tipo de bloqueo
- [ ] Exportación del historial en CSV para cumplimiento normativo
- [ ] El panel es accesible solo para usuarios con rol `admin` de la empresa

Definition of done: Panel con datos reales de la demo · exportación CSV funcional · todos los tipos de bloqueo registrados · acceso restringido a admin

---

#19 — Implementar dashboard de seguridad en tiempo real

Tipo: `feature` · **Prioridad:** `media` · **Estimado:** 5h

Acceptance criteria:

- [ ] El dashboard muestra en tiempo real: amenazas bloqueadas hoy · consultas fuera de contexto · accesos a columnas restringidas · activaciones del Circuit Breaker
- [ ] Las métricas se alimentan automáticamente desde los eventos de App Insights
- [ ] Gráfico de tendencia de amenazas por día de los últimos 7 días
- [ ] Clasificación de tipos de ataque detectados (encoding · role-play · injection · out-of-context)
- [ ] Comparación de tasa de bloqueo entre empresas (solo visible para super-admin)
- [ ] El dashboard está vinculado al panel de auditoría — hacer clic en una métrica filtra el historial

Definition of done: Dashboard en vivo con datos reales durante la demo · al menos 5 amenazas simuladas visibles · vínculo con auditoría funcional

---

M6 — Frontend Next.js + React

Chat · visualizaciones · auditoría · dashboard de seguridad · onboarding

Estimado total: 16 horas

#20 — Implementar interfaz de chat conversacional

Tipo: `ux` · **Prioridad:** `alta` · **Estimado:** 5h

Acceptance criteria:

- [ ] El usuario escribe una pregunta y ve la respuesta en menos de 15 segundos en el happy path
- [ ] El chat muestra el estado de cada agente en tiempo real: `Analizando → Generando SQL (LangGraph) → Ejecutando vía DAB → Generando insights`
- [ ] El SQL generado se muestra en bloque de código con syntax highlighting colapsable
- [ ] Los bloqueos de seguridad se muestran con un ícono de escudo y el motivo del bloqueo
- [ ] El usuario puede copiar el SQL con un click
- [ ] El historial de la conversación se mantiene durante la sesión para preguntas de seguimiento

Definition of done: Demo fluida sin tiempos muertos · estados de agentes visibles · bloqueos de seguridad visibles con motivo · SQL con syntax highlighting

---

#21 — Implementar visualizaciones automáticas con Chart.js

Tipo: `ux` · **Prioridad:** `alta` · **Estimado:** 4h

Acceptance criteria:

- [ ] El sistema selecciona automáticamente el tipo de gráfico según el tipo de dato retornado
- [ ] Se soportan al menos 4 tipos: barras, líneas, dispersión y tabla de datos
- [ ] Cada visualización muestra la justificación del tipo de gráfico citando el libro de Cole
- [ ] El usuario puede cambiar el tipo de gráfico manualmente si lo prefiere
- [ ] Los gráficos son responsive y se ven bien en pantalla de proyector

Definition of done: 4 tipos de chart funcionando · justificación del libro visible · cambio manual operativo · se ve bien en proyector

---

#22 — Implementar panel de trazabilidad visible para el usuario

Tipo: `ux` · **Prioridad:** `media` · **Estimado:** 3h

Acceptance criteria:

- [ ] Botón `¿Cómo llegué a este resultado?` despliega: intención detectada → SQL generado → endpoint DAB usado → filas retornadas → insight generado → fuente citada
- [ ] Si hubo bloqueos en el camino se muestran en el flujo con el motivo y el filtro que los detectó
- [ ] Las fuentes muestran libro · capítulo · página
- [ ] El tiempo total y por agente es visible para el usuario
- [ ] El panel es comprensible para un usuario de negocio sin conocimientos técnicos

Definition of done: Panel probado con 3 personas externas que entienden el razonamiento sin ayuda · bloqueos visibles en el flujo

---

#23 — Implementar pantalla de onboarding y conexión de empresa

Tipo: `ux` · **Prioridad:** `media` · **Estimado:** 4h

Acceptance criteria:

- [ ] Formulario de conexión con validación en tiempo real de la cadena de conexión
- [ ] Indicador de progreso: Conectando → Descubriendo esquema → Detectando columnas sensibles → Generando DAB config → Indexando → Listo
- [ ] El usuario puede revisar y ajustar qué columnas se marcan como sensibles antes de confirmar
- [ ] Al completar muestra 3 preguntas de ejemplo generadas del esquema real
- [ ] Proceso completo en menos de 5 minutos desde cero
- [ ] Botón para regenerar la configuración de DAB si el esquema cambia

Definition of done: Onboarding de empresa de prueba en menos de 5 minutos · columnas sensibles detectadas y ajustables · preguntas de ejemplo relevantes

---

M7 — Demo, datos y presentación

Contoso Data Generator · pruebas E2E · script de demo · pitch final

Estimado total: 10 horas

#24 — Generar datasets de demo con Contoso Data Generator

Tipo: `infra` · **Prioridad:** `alta` · **Estimado:** 3h

Acceptance criteria:

- [ ] Contoso Data Generator descargado desde `docs.sqlbi.com/contoso-data-generator`
- [ ] Dataset de ventas generado con tablas: `Sales`, `Customers`, `Products`, `Stores`, `Dates` (mínimo 50.000 filas en Sales)
- [ ] Dataset exportado en SQL Server via bulk-insert y cargado en Azure SQL
- [ ] Empresa B tiene BD de RRHH con tablas: `Employees`, `Departments`, `Salaries`, `Performance`
- [ ] Los datos incluyen tendencias y anomalías identificadas para usarlas en la demo del Agente 4
- [ ] La tabla `Employees` tiene columnas sensibles (`salary`, `ssn`) para demostrar los permisos DAB

Definition of done: Dos empresas con datos en Azure SQL · columnas sensibles en tabla Employees verificadas · anomalías identificadas y documentadas para la demo

---

#25 — Ejecutar pruebas de integración end-to-end y preparar demo

Tipo: `infra` · **Prioridad:** `alta` · **Estimado:** 5h

Acceptance criteria:

- [ ] Flujo completo (pregunta → SQL → DAB → insight) funciona en menos de 15 segundos promedio
- [ ] Las 20 preguntas de prueba retornan resultado correcto con SQL válido
- [ ] Prompt Shields bloquea los 15 intentos de ataque de prueba al 100%
- [ ] El filtro de contexto bloquea los 10 casos de acceso fuera de dominio al 100%
- [ ] El Circuit Breaker actúa correctamente al simular BD caída
- [ ] El dashboard de seguridad muestra todas las amenazas simuladas en tiempo real
- [ ] Los 13 servicios están activos y demostrables durante la presentación
- [ ] El script de demo está ensayado al menos 3 veces con el equipo completo

Definition of done: 20/20 pruebas pasando · tiempos dentro del umbral · dashboard de seguridad con amenazas simuladas visibles · script ensayado y cronometrado en 5 min

---

📊 Resumen de issues por módulo

Módulo	Issues	Estimado	Sprint
M1 — Infraestructura + .env	3 issues	10h	Días 1–2
M2 — Firebase Auth + multi-tenant	4 issues	19h	Días 3–4

M3 — RAG conocimiento	3 issues	11h	Días 5–6 (paralelo M2)
M4 — Agentes de IA + LangGraph	6 issues	31h	Días 7–10
M5 — Seguridad + auditoría + dashboard	3 issues	13h	Días 10–12
M6 — Frontend Next.js	4 issues	16h	Días 6–12 (paralelo)
M7 — Demo y pruebas	2 issues	10h	Días 12–13
Total	**25 issues**	**~110h**	**13 días**

🔖 Etiquetas de GitLab

Etiqueta	Color	Uso
`feature`	Azul	Nueva funcionalidad del sistema
`seguridad`	Rojo	Issues de seguridad y filtros
`infra`	Verde	Infraestructura, CI/CD, servicios
`ux`	Naranja	Frontend y experiencia de usuario
`prioridad:alta`	Rojo	Bloquea avance si no está hecho
`prioridad:media`	Amarillo	Importante pero no bloquea el núcleo
`dab`	Morado	Issues de Data API Builder
`langgraph`	Celeste	Issues del grafo de agentes

✅ Definition of Done global del proyecto

- [ ] Los 4 agentes funcionan en secuencia en el happy path sin errores
- [ ] Firebase Auth retorna token con custom claims correctos por usuario y empresa
- [ ] DAB corre con permisos diferenciados por rol — analyst no ve columnas sensibles
- [ ] Dos empresas distintas conectadas y operativas de forma aislada
- [ ] Los dos libros indexados y RAG retorna chunks relevantes via SDK en código
- [ ] Prompt Shields bloquea los 15 intentos de ataque al 100%
- [ ] Filtro de contexto bloquea los 10 casos de acceso fuera de dominio al 100%

- [ ] Circuit Breaker actúa correctamente al simular BD caída
- [ ] Dashboard de seguridad muestra amenazas en tiempo real con datos reales
- [ ] Panel de auditoría con historial completo y exportación CSV funcional
- [ ] Los 13 servicios están activos y demostrables
- [ ] El tiempo promedio de respuesta es menor a 15 segundos
- [ ] El frontend se ve correctamente en la pantalla del proyector
- [ ] El script de demo ensayado 3 veces con el equipo

📎 Referencias técnicas del proyecto

Recurso	Referencia
Data API Builder	`dab --help` · `dab init` · `dab add` · `/swagger` · `/health`
Firebase Auth Admin SDK	`firebase-admin` Python · `verifyIdToken()` · custom claims
Prompt Shields API	Azure AI Content Safety · Prompt Shields endpoint
LangGraph	`langgraph` Python · StateGraph · nodos y aristas condicionales
Azure AI Search SDK	`azure-search-documents` · `SearchClient` · `VectorizedQuery`
Circuit Breaker	`pybreaker` Python · `CircuitBreaker(fail_max=5, reset_timeout=60)`
Contoso Data Generator	`docs.sqlbi.com/contoso-data-generator`
OpenTelemetry Python	`azure-monitor-opentelemetry` SDK

🔄 Cambios respecto a v2

Componente	v2	v3
Autenticación	Azure Entra ID	Firebase Auth con custom claims
Secretos	Azure Key Vault	.env por empresa en Container Apps
Permisos BD	Solo read-only global	Permisos granulares por rol y columna en DAB
Agente 2	Reintentos manuales	LangGraph con grafo de estados y backoff exponencial

IA Responsable	RAI Toolbox dashboard	Filtros de contexto + dashboard de amenazas en tiempo real
Flujo de aprobación	Azure Durable Functions	Eliminado — reemplazado por filtros automáticos + auditoría
Servicios totales	15	13 (más enfocados y demostrables)
Issues totales	27	25 (sin issues descartados)

Documento v3 — Firebase Auth · .env · DAB permisos por rol · LangGraph · Filtros de contexto · Dashboard de seguridad · Circuit Breaker · Sin flujo de aprobación manualHackaton Microsoft Azure · 13 servicios · 4 agentes · Semantic Kernel + LangGraph · RAG con 2 libros · 25 issues · ~110h estimadas